Guía paso a paso — Sistema IoT de Seguridad (Detección y alertas)

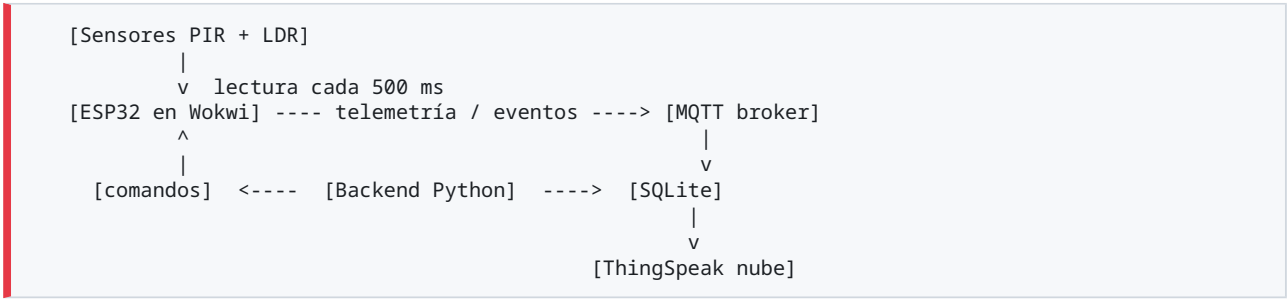
Parcial #2 · Desarrollo de Software VIII · Universidad Tecnológica de Panamá
Escenario elegido: 4) Sistema de Seguridad - Detección y alertas
Autor: Richard Rodríguez — github.com/Death272004

---

0. Qué vas a construir

Un sistema de seguridad completo que:

1. **Detecta movimiento** con un sensor PIR y mide la **luz ambiental** con un LDR.
2. **Activa alarmas**: LED rojo parpadeante, buzzer (sirena) y pantalla OLED con el estado.
3. **Se arma y desarma** desde un botón físico o **remotamente desde la nube**.
4. **Publica telemetría y eventos por MQTT** desde un ESP32 (simulado en Wokwi).
5. **Guarda todo en SQLite** con fecha y hora: lecturas, eventos y comandos ejecutados.
6. **Muestra un dashboard en tiempo real en ThingSpeak** y recibe **comandos desde la nube**.



---

1. Requisitos previos

#	Requisito	Detalle
1	Cuenta en **Wokwi**	https://wokwi.com (plan gratuito)
2	Cuenta en **ThingSpeak**	https://thingspeak.com (plan gratuito)
3	**Python 3.9 o superior**	`python3 --version` en una terminal
4	**Git** y cuenta en **GitHub**	para subir el código fuente
5	Este repositorio	`Parcial2-Sistema-Seguridad-IoT`

> **Dato importante:** Wokwi tiene **acceso real a Internet** desde la simulación (IoT Gateway). Por eso el ESP32 puede conectarse a `broker.hivemq.com` sin instalar ningún broker local.

---

2. Estructura del proyecto

```
Parcial2-Sistema-Seguridad-IoT/
├── README.md                  Documentación principal
├── requirements.txt           Dependencias del backend
├── firmware/                  Código del dispositivo (Wokwi)
│   ├── main.py                Bucle principal / orquestador
│   ├── config.py              Pines, red, MQTT, umbrales
│   ├── sensors.py             PIR + LDR (conversión a lux)
│   ├── actuators.py           LEDs, buzzer y panel de alarma
│   ├── display.py             Pantalla OLED SSD1306
│   ├── net_mqtt.py            WiFi + cliente MQTT
│   ├── security.py            Máquina de estados y eventos
│   ├── diagram.json           Circuito de Wokwi
│   └── libraries.txt          Bibliotecas de Wokwi
├── backend/                   Integración Python 3
│   ├── main.py                Orquestador (MQTT ↔ SQLite ↔ ThingSpeak)
│   ├── config.py              Configuración / variables de entorno
│   ├── database.py            Capa SQLite (thread-safe)
│   ├── mqtt_bridge.py         Puente MQTT (paho 1.x / 2.x)
│   ├── thingspeak.py          Cliente ThingSpeak + TalkBack
│   ├── queries.py             CLI de históricos
│   └── cli.py                 Envío manual de comandos
├── sql/
│   ├── schema.sql             Esquema de la base de datos
│   └── consultas.sql          Consultas de histórico comentadas
├── tests/
│   └── simulador_dispositivo.py   Emula el ESP32 para probar sin Wokwi
└── docs/
    ├── arquitectura.md        Diagramas, contratos y modelo de datos
    ├── guia-paso-a-paso.md    Este archivo (fuente del PDF)
    └── capturas/              Capturas de pantalla (entregable)
```

---

3. PARTE A — Montar el circuito en Wokwi

A.1 Crear el proyecto

1. Entra a https://wokwi.com y pulsa **Start** → elige **ESP32 + MicroPython**.
2. En la pestaña **Diagram**, borra el contenido por defecto y pega el archivo `firmware/diagram.json` de este repositorio.

A.2 Componentes utilizados

#	Componente	Parte en Wokwi	Cantidad
1	Placa ESP32 DevKit V4	`board-esp32-devkit-c-v4`	1
2	Sensor de movimiento PIR	`wokwi-pir-motion-sensor`	1
3	Módulo fotoresistivo (LDR)	`wokwi-photoresistor-sensor`	1
4	LED rojo (alarma)	`wokwi-led`	1
5	LED verde (estado)	`wokwi-led`	1
6	Buzzer	`wokwi-buzzer`	1
7	Pantalla OLED 128×64	`board-ssd1306`	1
8	Botón táctil	`wokwi-pushbutton`	1
9	Resistencias 220 Ω	`wokwi-resistor`	2

A.3 Tabla de cableado

Señal	Componente	Pin ESP32	Color sugerido
Entrada de movimiento	PIR → `OUT`	**GPIO 13**	naranja
Alimentación PIR	PIR → `VCC` / `GND`	3V3 / GND	rojo / negro
Entrada analógica	LDR → `AO`	**GPIO 34** (ADC)	amarillo
Alimentación LDR	LDR → `VCC` / `GND`	3V3 / GND	rojo / negro
Alarma visual	LED rojo → R 220 Ω	**GPIO 25**	rojo
Indicador ARMADO	LED verde → R 220 Ω	**GPIO 27**	verde
Sirena	Buzzer pin 2 (+)	**GPIO 26**	morado
Cátodos	LED rojo/verde `C`, Buzzer pin 1	GND	negro
Botón	Pushbutton `1.l`	**GPIO 14**	azul
Botón	Pushbutton `2.l`	GND	negro
Display I2C	OLED → `SDA` / `SCL`	**GPIO 21 / 22**	cian / magenta
Alimentación OLED	OLED → `VCC` / `GND`	3V3 / GND	rojo / negro
Monitor serie	ESP32 `TX` / `RX`	`$serialMonitor`	—

> El botón se conecta a GND y usa la **resistencia de pull-up interna** del ESP32 (se detecta el flanco de bajada, sin componente externo).

A.4 Bibliotecas de Wokwi

En la pestaña de código, abre el **Library Manager** y agrega:

```
micropython-umqtt.simple
micropython-ssd1306
```

(Equivalente a pegar el contenido de `firmware/libraries.txt`.)

---

4. PARTE B — Código del dispositivo (firmware)

B.1 Cargar los archivos

En el editor de Wokwi, junto a la pestaña `main.py` pulsa el botón **+** y crea los archivos `config.py`, `sensors.py`, `actuators.py`, `display.py`, `net_mqtt.py` y `security.py`, pegando el contenido correspondiente.

Wokwi copia **todos los archivos del proyecto** al sistema de archivos del ESP32, por lo que los `import` entre módulos funcionan igual que en Python normal.

B.2 Qué hace cada módulo

Archivo	Responsabilidad	Elementos clave
`config.py`	**Todo** lo ajustable en un solo lugar	pines, SSID, broker, tópicos, umbrales, temporizaciones
`sensors.py`	Lee el hardware	`MotionSensor` (PIR), `LightSensor` (LDR → lux)
`actuators.py`	Controla salidas sin bloquear	`Led`, `Buzzer` (PWM), `AlarmPanel.tick()`
`display.py`	Presentación	`OledDisplay.render(snapshot)` con degradación elegante
`net_mqtt.py`	Comunicación	`connect_wifi()`, `sync_time()` (NTP), `MqttService`
`security.py`	**Dominio / lógica de negocio**	`SecuritySystem`: estados, eventos y comandos
`main.py`	Orquestación	bucle de muestreo y despacho de eventos

B.3 El bucle principal (orientado a eventos, no bloqueante)

```
while True:
    now = time.ticks_ms()
    mqtt.check_messages()                    # 1. comandos desde la nube
    for payload in mqtt.drain_commands():
        system.handle_command(payload, "mqtt")
    ... botón local ...                      # 2. armado/desarmado físico
    if vencio SAMPLE_MS:                     # 3. lectura de sensores
        system.update(pir.read(), ldr.read_lux(), now)
    alarm_panel.tick(now)                    # 4. parpadeo y pitido
    if vencio TELEMETRY_MS:                  # 5. publicación
        mqtt.publish_telemetry(system.snapshot())
    if vencio DISPLAY_MS:                    # 6. pantalla
        display.render(system.snapshot())
    mqtt.ensure_connected()                  # 7. reconexión
    time.sleep_ms(50)
```

Nada de lo anterior espera la red: `check_msg()` retorna de inmediato si no hay mensajes, y los tiempos se comparan con `time.ticks_ms()`.

B.4 Máquina de estados

```
  DESARMADO --(ARM/boton)--> ARMADO --(movimiento)--> ALARMA
       ^                       ^                       |
       |                       |                       |
       +---(DISARM/boton)------+<---(SILENCE/30 s)------+
       |                                               |
       +---------------(DISARM/boton)----------------+
```

Cada transición genera un **evento** (`ARMED`, `MOTION_DETECTED`, `ALARM_ON`, `ALARM_SILENCED`, `ALARM_TIMEOUT`, `COMMAND_RECEIVED`, ...) que se publica por MQTT y se guarda en SQLite.

B.5 Comandos aceptados

Comando	Acción
`ARM` / `ARMAR`	activa la vigilancia (enciende LED verde)
`DISARM` / `DESARMAR`	desactiva la vigilancia
`SILENCE` / `SILENCIAR`	silencia la alarma manteniendo el sistema armado
`STATUS` / `ESTADO`	reporta el estado actual
`PING`	comprueba que el dispositivo responde

El parser acepta **texto plano** (`ARM`) o **JSON** (`{"cmd":"ARM"}`).

---

5. PARTE C — Backend en Python

C.1 Instalación

```
cd Parcial2-Sistema-Seguridad-IoT
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
```

Dependencias: `paho-mqtt` (cliente MQTT) y `requests` (HTTP hacia ThingSpeak). `sqlite3` forma parte de la biblioteca estándar.

C.2 Ejecutar

```
python -m backend.main
```

Salida esperada:

```
============================================================
 BACKEND - Sistema de Seguridad IoT
 MQTT  broker.hivemq.com:1883 | SQLite .../data/seguridad.db
 ThingSpeak dashboard: deshabilitado (falta THINGSPEAK_WRITE_KEY)
 ThingSpeak TalkBack : deshabilitado (faltan claves de TalkBack)
============================================================
[mqtt] conectado a broker.hivemq.com:1883 y suscrito a:
        - seguridad/casa-01/telemetry
        - seguridad/casa-01/event
```

La base `data/seguridad.db` se crea automáticamente con `sql/schema.sql`.

C.3 Prueba sin Wokwi (recomendada antes de la simulación)

```
python -m tests.simulador_dispositivo --segundos 30 --movimiento
```

Este script hace exactamente lo que haría el ESP32: publica telemetría cada 2 s, genera una intrusión a los 12 segundos y responde a los comandos. Permite validar **toda la cadena broker → backend → SQLite** en un minuto.

---

6. PARTE D — ThingSpeak (plataforma en la nube)

D.1 Crear el canal

1. Entra a https://thingspeak.com → **New Channel**.
2. Nombre: `Sistema de Seguridad IoT`.
3. Habilita cuatro campos:

Campo	Nombre	Contenido
Field 1	Movimiento	0 / 1
Field 2	Luz (lux)	iluminación ambiental
Field 3	Armado	0 / 1
Field 4	Alarma	0 / 1

1. Pestaña **API Keys**: copia `Channel ID`, **Write API Key** y **Read API Key**.

D.2 Habilitar el dashboard

```
export THINGSPEAK_CHANNEL_ID="1234567"
export THINGSPEAK_WRITE_KEY="XXXXXXXXXXXXXXXX"
python -m backend.main
```

El backend envía una actualización cada **15 s** (límite del plan gratuito) y el dashboard de ThingSpeak muestra las cuatro gráficas en tiempo real, más el campo de estado con la descripción del sistema.

D.3 Comandos desde la nube (TalkBack)

1. En ThingSpeak: **Apps** → **TalkBack** → **New TalkBack** (vinculado al canal).
2. Copia **TalkBack ID** y **TalkBack API Key**:

```
export THINGSPEAK_TALKBACK_ID="987654"
export THINGSPEAK_TALKBACK_KEY="ZZZZZZZZZZZZZZZZ"
python -m backend.main
```

1. En la pestaña **Commands** de TalkBack escribe, por ejemplo, `ARM` y guarda.
2. El backend toma el comando por REST cada 10 s, lo reenvía por MQTT al dispositivo y el dispositivo responde con el evento `COMMAND_RECEIVED`.

Alternativa sin TalkBack: usar un cliente MQTT web (por ejemplo https://www.hivemq.com/demos/websocket-client/), conectarte a `broker.hivemq.com` y publicar en `seguridad/casa-01/command` con payload `ARM`.

---

7. PARTE E — Base de datos y consultas de histórico

E.1 Esquema

Tabla	Contenido
`lecturas`	una fila por cada telemetría: `recibido_at`, `motion`, `lux`, `armed`, `alarm`…
`eventos`	eventos de seguridad: `MOTION_DETECTED`, `ALARM_ON`, `COMMAND_RECEIVED`…
`comandos`	trazabilidad: `origen`, `comando`, `estado` (`ENVIADO → APLICADO / RECHAZADO`)
`vista_ultima_lectura`	última lectura registrada
`vista_alarmas`	histórico de alarmas y detecciones

El script completo está en `sql/schema.sql`.

E.2 Consultas desde Python

```
python -m backend.queries resumen
python -m backend.queries lecturas -n 20
python -m backend.queries eventos -n 30
python -m backend.queries eventos --tipo MOTION_DETECTED
python -m backend.queries comandos -n 10
python -m backend.queries entre --desde 2026-11-08T00:00:00 --hasta 2026-11-08T23:59:59
```

E.3 Consultas SQL directas

```
sqlite3 data/seguridad.db < sql/consultas.sql
```

Incluye diez consultas comentadas: últimas lecturas, historial de intrusiones, histórico de alarmas, trazabilidad de comandos, rango de fechas, estadísticas de luz, distribución de eventos, intrusiones con sistema armado, última lectura y resumen general.

E.4 Enviar comandos desde la terminal

```
python -m backend.cli ARM
python -m backend.cli SILENCE
python -m backend.cli --list
```

---

8. PARTE F — Secuencia completa de demostración

Paso	Acción	Qué debe observarse
1	`python -m backend.main`	el backend se suscribe a los tópicos
2	▶ **Start Simulation** en Wokwi	el OLED muestra *Iniciando...* y luego el estado
3	Mover el cursor sobre el sensor PIR	el LED verde parpadea y aparece `MOTION_DETECTED`
4	Cambiar el valor de `lux` del LDR	cambian `lux`, `light_state` y la gráfica 2 de ThingSpeak
5	Pulsar la tecla configurada (`b`) o el botón	`ARMED` → LED verde encendido
6	Generar movimiento estando ARMADO	**alarma**: LED rojo parpadea + buzzer + `ALARM_ON`
7	`python -m backend.cli SILENCE`	la alarma se silencia y sigue armado
8	`python -m backend.queries resumen`	conteos de lecturas, eventos y comandos
9	ThingSpeak → **TalkBack** → **Commands** → `ARM`	el comando llega al dispositivo por MQTT
10	Capturas de pantalla	ver la sección de entregables

---

9. PARTE G — Entregables

1. **Código fuente en GitHub** (este repositorio) y copia en **Teams**.
2. **`README.md`** con descripción, arquitectura y instrucciones.
3. **`requirements.txt`** con las librerías utilizadas.
4. **Scripts SQL**: `sql/schema.sql` y `sql/consultas.sql`.
5. **Diagrama de la arquitectura**: `docs/arquitectura.md` (mermaid + ASCII).
6. **Capturas del circuito** en `docs/capturas/`:

Archivo	Qué muestra
`01-circuito-wokwi.png`	circuito completo con la simulación corriendo
`02-senal-alarma.png`	PIR activado con LED rojo y buzzer en marcha
`03-display-oled.png`	OLED mostrando ARMADO / ALARMA
`04-mqtt-messages.png`	telemetría y eventos en un cliente MQTT
`05-thingspeak-dashboard.png`	dashboard con las 4 gráficas en tiempo real
`06-sqlite-consultas.png`	salida de `backend.queries`
`07-comando-nube.png`	comando creado en la nube y ejecutado por el dispositivo
`08-arquitectura.png`	diagrama de arquitectura / flujo de datos

1. **Proyecto ZIP de Wokwi**: en Wokwi, menú **…** → **Download project files** (o *Share* → *Download*). Guarda el `.zip` en el repositorio.
2. **Documentación** (este PDF) y **defensa oral**.

Mapeo con los criterios de evaluación

Criterio	Peso	Dónde se cumple
Funcionamiento completo	30%	simulación Wokwi + backend + SQLite + ThingSpeak
Calidad de código	10%	7 módulos modulares, docstrings, PEP 8, sin código muerto
Integración	5%	MQTT ↔ SQLite ↔ ThingSpeak, trazabilidad de comandos
Documentación	5%	README, `docs/arquitectura.md`, este PDF, scripts SQL
Defensa oral	50%	ver la sección siguiente

---

10. Defensa oral — preguntas frecuentes

¿Por qué MQTT y no HTTP? MQTT es un protocolo ligero publica/suscribe diseñado para IoT: poca cabecera, funciona con conexiones inestables, permite tópicos jerárquicos y que múltiples suscriptores reciban el mismo evento sin duplicar peticiones.

¿Por qué un broker público? Wokwi entrega conectividad real al ESP32, así que se puede usar `broker.hivemq.com` sin instalar nada. En producción se usaría un broker propio (Mosquitto/EMQX).

¿Qué significa "orientado a eventos"? La lógica no se ejecuta de forma lineal: los cambios (llegada de un comando, flanco del PIR, timeout de la alarma) generan eventos que se despachan a escritores interesados (`emit()` → MQTT → base de datos → dashboard).

¿Por qué el bucle es no bloqueante? Si se usara `wait_msg()` o `sleep()` largo, el ESP32 dejaría de leer sensores y de atender la alarma. Con `check_msg()` y comparación de `ticks_ms()` el sistema responde en menos de 100 ms.

¿Dónde corre SQLite y por qué? En el backend (PC), porque MicroPython en el ESP32 no incluye SQLite. El dispositivo sólo manda datos; la persistencia y las consultas se hacen en el lado servidor, que sí tiene ficheros y más recursos.

¿Cómo se sincronizan los tiempos? El firmware hace sync con un servidor NTP (`ntptime`) y publica el timestamp; el backend agrega `recibido_at` con la hora local, de modo que siempre hay fecha y hora en la base aunque el dispositivo no logre sincronizar.

¿Cómo llega un comando desde la nube hasta el ESP32? ThingSpeak TalkBack → REST (`POST /talkbacks/<id>/commands/execute.json`) → backend Python → publicación MQTT en `seguridad/casa-01/command` → callback del dispositivo → `SecuritySystem.handle_command()` → evento `COMMAND_RECEIVED` que vuelve al backend para marcar el comando como `APLICADO`.

¿Qué pasa si se cae la red o el broker? `ensure_connected()` reintenta cada 5 s con retroceso, el display muestra el error, y el sistema sigue funcionando localmente (lecturas, botón y alarma).

¿Cómo evitas los rebotes del botón? Debouncing por software: se registra el último instante de pulsación y sólo se acepta un nuevo flanco pasados `BUTTON_DEBOUNCE_MS` (250 ms).

¿Por qué dos LEDs y no uno? Para separar semántica: verde = sistema armado (vigilancia activa), rojo = alarma activa. Así se comprueba de un vistazo el estado sin mirar la pantalla.

¿Cómo mediste la iluminación en lux? Se convierte el valor del ADC a tensión, luego a resistencia del LDR con el divisor de tensión del módulo y finalmente a lux con la fórmula `lux = (RL10 · 10³ · 10^γ / R)^(1/γ)` (γ = 0.7, RL10 = 50 kΩ), tomada de la documentación del módulo.

¿Por qué 15 segundos en ThingSpeak? El plan gratuito de ThingSpeak permite una actualización cada 15 s; el backend espacía los envíos para no perder datos por límite de tasa.

---

11. Solución de problemas

Síntoma	Causa probable	Solución
`ImportError: umqtt`	falta la biblioteca	agrega `micropython-umqtt.simple` en el Library Manager
`ImportError: ssd1306`	falta la biblioteca	agrega `micropython-ssd1306`; el sistema sigue sin OLED
El OLED queda en blanco	dirección I2C incorrecta	revisa `i2cAddress: "0x3c"` y los pines 21/22
`[wifi] ERROR: sin respuesta`	SSID incorrecto	usa `Wokwi-GUEST` (no pide contraseña)
`[mqtt] ERROR al conectar`	sin Internet en Wokwi	revisa que la simulación esté corriendo
No llega telemetría al backend	mismo `client_id` en dos procesos	usa IDs distintos (el CLI ya usa `-cli`)
`SQLite … thread`	conexión usada desde otro hilo	ya resuelto: `check_same_thread=False` + candado
ThingSpeak no recibe datos	falta Write API Key	exporta `THINGSPEAK_WRITE_KEY`
TalkBack no llega	faltan credenciales o cola vacía	exporta `THINGSPEAK_TALKBACK_ID` y `_KEY`, crea un comando
`No se pudo encontrar el esquema`	ejecución fuera de la raíz	corre `python -m backend.main` desde la raíz del repo
Alarma que no suena	el buzzer necesita PWM	`AlarmPanel` usa `machine.PWM` a 2 kHz; revisa `mode: "smooth"`

---

12. Comandos de referencia rápida

```
# Backend
python -m backend.main                    # levantar el backend
python -m backend.cli ARM|DISARM|SILENCE|STATUS|PING
python -m backend.queries resumen|lecturas|eventos|comandos|entre

# Pruebas
python -m tests.simulador_dispositivo --segundos 30 --movimiento

# Base de datos
sqlite3 data/seguridad.db < sql/consultas.sql

# Git
git add . && git commit -m "mensaje" && git push
```

Parcial #2 · Desarrollo de Software VIII · Universidad Tecnológica de Panamá · **Richard Rodríguez** · github.com/Death272004